

---

# TRÍ TUỆ NHÂN TẠO

Machine Learning

Phần 4 — Regression Analysis

Biên soạn:

**ĐẶNG TẤN QUÍ**

---

SĐT: 0377 122 210

2026

# Mục lục

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>Lời nói đầu</b>                                            | <b>4</b>  |
| <b>1 Phân tích hồi quy (Regression Analysis)</b>              | <b>5</b>  |
| 1.1 Thuật ngữ                                                 | 5         |
| 1.2 Cách hồi quy hoạt động                                    | 5         |
| 1.3 Các loại hồi quy trong học máy                            | 6         |
| 1.3.1 Hồi quy tuyến tính (Linear Regression)                  | 6         |
| 1.3.2 Hồi quy logistic (Logistic Regression)                  | 6         |
| 1.3.3 Hồi quy đa thức (Polynomial Regression)                 | 7         |
| 1.3.4 Hồi quy Lasso (Lasso Regression)                        | 7         |
| 1.3.5 Hồi quy Ridge (Ridge Regression)                        | 7         |
| 1.3.6 Hồi quy cây quyết định (Decision Tree Regression)       | 8         |
| 1.3.7 Hồi quy Random Forest (Random Forest Regression)        | 8         |
| 1.3.8 Hồi quy vectơ hỗ trợ (Support Vector Regression)        | 8         |
| 1.4 Hai loại mô hình hồi quy (đơn và đa biến)                 | 9         |
| 1.5 Chọn mô hình hồi quy tốt nhất                             | 9         |
| 1.6 Metric đánh giá hồi quy                                   | 9         |
| 1.7 Ứng dụng hồi quy trong học máy                            | 10        |
| 1.8 Xây dựng mô hình hồi quy trong Python                     | 10        |
| <b>2 Hồi quy tuyến tính (Linear Regression)</b>               | <b>12</b> |
| 2.1 Khái niệm chi tiết                                        | 12        |
| 2.2 Quan hệ tuyến tính dương và âm                            | 13        |
| 2.3 Hai loại hồi quy tuyến tính                               | 14        |
| 2.3.1 Hồi quy tuyến tính đơn (Simple Linear Regression)       | 14        |
| 2.3.2 Hồi quy tuyến tính đa biến (Multiple Linear Regression) | 15        |
| 2.4 Cách hồi quy tuyến tính hoạt động                         | 15        |
| 2.5 Hàm giả thuyết                                            | 16        |
| 2.6 Đường khớp tốt nhất (Finding the Best Fit Line)           | 16        |
| 2.7 Hàm mất mát cho hồi quy tuyến tính                        | 17        |
| 2.8 Gradient descent cho tối ưu                               | 17        |
| 2.9 Giả định của hồi quy tuyến tính                           | 17        |
| 2.10 Metric đánh giá                                          | 18        |
| 2.11 Ứng dụng hồi quy tuyến tính                              | 18        |
| 2.12 Ưu điểm hồi quy tuyến tính                               | 19        |
| 2.13 Thách thức thường gặp                                    | 19        |
| 2.14 Hồi quy đa thức — phương án thay thế                     | 19        |
| <b>3 Hồi quy tuyến tính đơn (Simple Linear Regression)</b>    | <b>20</b> |
| 3.1 Biến độc lập và phụ thuộc                                 | 20        |
| 3.2 Biểu diễn đồ thị                                          | 21        |
| 3.3 Mô hình toán học                                          | 21        |
| 3.4 Cách hồi quy tuyến tính đơn hoạt động                     | 21        |
| 3.5 Giả định                                                  | 22        |
| 3.6 Triển khai bằng Python                                    | 22        |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>4 Hồi quy tuyến tính đa biến (Multiple Linear Regression)</b> | <b>27</b> |
| 4.1 Khái niệm chi tiết . . . . .                                 | 27        |
| 4.2 Phương trình . . . . .                                       | 28        |
| 4.3 Giả định . . . . .                                           | 28        |
| 4.4 Triển khai bằng Python (Scikit-learn) . . . . .              | 29        |
| 4.5 Ứng dụng . . . . .                                           | 31        |
| 4.6 Thách thức . . . . .                                         | 32        |
| 4.7 So sánh hồi quy đơn và đa biến . . . . .                     | 32        |
| <b>5 Hồi quy đa thức (Polynomial Regression)</b>                 | <b>33</b> |
| 5.1 Vì sao cần hồi quy đa thức? . . . . .                        | 33        |
| 5.2 Phương trình mô hình . . . . .                               | 34        |
| 5.3 Cách hoạt động . . . . .                                     | 34        |
| 5.4 Triển khai bằng Python (Scikit-learn) . . . . .              | 34        |

Dặng Tấn Quý

## Lời nói đầu

### Nguồn

Tài liệu biên soạn và dịch đầy đủ từ Tutorialspoint Machine Learning — mục Regression Analysis: [https://www.tutorialspoint.com/machine\\_learning/machine\\_learning\\_regression\\_analysis.htm](https://www.tutorialspoint.com/machine_learning/machine_learning_regression_analysis.htm).

### Cách dùng

- Đọc sau phần Statistics và Data Visualization.

Dặng Tấn Quý

# 1 Phân tích hồi quy (Regression Analysis)

## Phân tích hồi quy

Kỹ thuật thống kê dự đoán **giá trị số liên tục** dựa trên quan hệ giữa biến độc lập và biến phụ thuộc.

Mục tiêu: vẽ đường/đường cong khớp dữ liệu tốt nhất và ước lượng ảnh hưởng của biến này lên biến kia.

## Vai trò trong học máy

Hồi quy là **học có giám sát**: mô hình học từ feature (độc lập) và target (phụ thuộc, số liên tục) để dự đoán trên dữ liệu mới.

Ứng dụng: dự báo, phân tích dự đoán, tối ưu kinh doanh, kinh tế, tài chính.

## 1.1 Thuật ngữ

### Thuật ngữ thường gặp

- **Biến độc lập** (predictor, biến dự báo, feature): dùng để dự đoán giá trị biến phụ thuộc.
- **Biến phụ thuộc** (target, biến mục tiêu): biến cần dự đoán.
- **Đường hồi quy**: đường thẳng / đường cong phù hợp nhất với dữ liệu.
- **Quá khớp (Overfitting)** (phương sai cao): mô hình tốt trên tập huấn luyện nhưng kém trên tập kiểm tra.
- **Thiếu khớp (Underfitting)** (độ lệch cao): mô hình kém cả trên tập huấn luyện.
- **Điểm ngoại lai (Outlier)**: điểm không theo quy luật chung; giá trị cực cao hoặc cực thấp.
- **Đa cộng tuyến (Multicollinearity)**: các biến độc lập tương quan cao với nhau.

## 1.2 Cách hồi quy hoạt động

### Quy trình

Thuật toán hồi quy được huấn luyện trên tập có nhãn (feature + target).

Trong huấn luyện, mô hình học quan hệ giữa feature và target; sau đó dự đoán giá trị mới từ quan hệ đã học.

## 1.3 Các loại hồi quy trong học máy

### Cách phân loại và các loại thường dùng

Nói chung, các phương pháp hồi quy được phân loại dựa trên ba tiêu chí: số biến độc lập, kiểu biến phụ thuộc, và hình dạng đường hồi quy.

### 1.3.1 Hồi quy tuyến tính (Linear Regression)

#### Hồi quy tuyến tính

Hồi quy tuyến tính là mô hình hồi quy được dùng phổ biến nhất trong học máy. Đây là mô hình thống kê phân tích **quan hệ tuyến tính** giữa biến phụ thuộc với một tập biến độc lập cho trước.

#### Quan hệ tuyến tính và hai nhánh

Quan hệ tuyến tính giữa các biến nghĩa là: khi giá trị của một hoặc nhiều biến độc lập thay đổi (tăng hoặc giảm), giá trị biến phụ thuộc cũng thay đổi tương ứng (tăng hoặc giảm).

Hồi quy tuyến tính chia thành hai nhánh con:

- **Hồi quy tuyến tính đơn** (simple linear regression): chỉ dùng **một** biến độc lập (predictor) để dự đoán biến phụ thuộc.
- **Hồi quy tuyến tính đa biến** (multiple linear regression): nhiều biến độc lập được dùng để dự đoán biến phụ thuộc.

### 1.3.2 Hồi quy logistic (Logistic Regression)

#### Hồi quy logistic

Hồi quy logistic là thuật toán học máy phổ biến, dùng để dự đoán **xác suất** một sự kiện xảy ra.

#### Mô hình và hàm sigmoid

Hồi quy logistic là mô hình tuyến tính tổng quát (generalized linear model) trong đó biến mục tiêu tuân theo phân phối Bernoulli. Thuật toán dùng hàm logistic hoặc hàm logit để học quan hệ giữa biến độc lập (predictors) và biến phụ thuộc (target).

Biến phụ thuộc được ánh xạ qua hàm sigmoid của các biến độc lập. Hàm sigmoid cho xác suất trong khoảng  $[0, 1]$ ; giá trị xác suất này được dùng để ước lượng giá trị biến phụ thuộc.

#### Ứng dụng chính

Hồi quy logistic chủ yếu dùng cho **bài toán phân loại nhị phân**: biến mục tiêu là phân loại với hai lớp. Mô hình ước lượng xác suất của biến mục tiêu khi biết các feature đầu

vào, rồi dự đoán lớp có xác suất cao nhất.

### 1.3.3 Hồi quy đa thức (Polynomial Regression)

#### Hồi quy đa thức

Polynomial Linear Regression là phân tích hồi quy trong đó quan hệ giữa biến độc lập và biến phụ thuộc được mô hình hóa bằng hàm đa thức bậc  $n$ . Hồi quy đa thức cho phép bất quan hệ phức tạp hơn so với quan hệ tuyến tính trong hồi quy đơn và hồi quy tuyến tính đa biến thuần túy.

#### Đặc điểm

Hồi quy đa thức là một trong các hồi quy **phi tuyến** được dùng rộng rãi nhất. Có thể mô hình hóa quan hệ phi tuyến giữa predictor và target; đồng thời **nhạy hơn với outlier** so với hồi quy tuyến tính.

### 1.3.4 Hồi quy Lasso (Lasso Regression)

#### Hồi quy Lasso

Hồi quy Lasso là kỹ thuật **chuẩn hóa** (regularization): thêm hình phạt (penalty) để tránh overfitting và cải thiện độ chính xác mô hình hồi quy.

#### L1 và trường hợp dùng

Lasso thực hiện **chuẩn hóa L1**: sửa hàm mất mát bằng cách cộng thêm lượng co (shrinkage) bằng **tổng giá trị tuyệt đối** của các hệ số.  
Hồi quy Lasso thường dùng khi xử lý dữ liệu **chiều cao** (nhiều feature) và dữ liệu có **tương quan cao** giữa các biến.

### 1.3.5 Hồi quy Ridge (Ridge Regression)

#### Hồi quy Ridge

Hồi quy Ridge là kỹ thuật thống kê trong học máy nhằm **ngăn overfitting** trên mô hình hồi quy tuyến tính.

#### L2 và trường hợp dùng

Ridge là chuẩn hóa **L2**: sửa hàm mất mát / cost bằng cách cộng penalty bằng **bình phương độ lớn** của các hệ số.  
Ridge giúp giảm độ phức tạp mô hình và cải thiện độ dự đoán. Phù hợp khi mô hình có nhiều tham số với trọng số lớn, và với tập dữ liệu có **số feature lớn hơn số quan sát**.

### Đa cộng tuyến

Ridge cũng giúp xử lý **đa cộng tuyến** — tình huống các biến độc lập phụ thuộc lẫn nhau.

### 1.3.6 Hồi quy cây quyết định (Decision Tree Regression)

#### Hồi quy cây quyết định

Hồi quy cây quyết định dùng thuật toán cây quyết định để dự đoán **giá trị số**. Cây quyết định là thuật toán học có giám sát, dùng được cho cả phân loại và hồi quy.

#### Cách hoạt động

Trong hồi quy, cây dùng để dự đoán biến liên tục. Cách hoạt động: chia dữ liệu thành các tập con nhỏ hơn theo giá trị feature đầu vào, gán mỗi tập con một giá trị số; quá trình lặp dần xây dựng cây quyết định.

Cây khớp các hồi quy tuyến tính cục bộ để xấp xỉ đường cong; mỗi lá (leaf) biểu diễn một giá trị số. Thuật toán cố gắng **giảm MSE** tại mỗi nút con — đo mức dự đoán lệch so với target gốc.

#### Ứng dụng

Dự đoán giá cổ phiếu, hành vi khách hàng, v.v.

### 1.3.7 Hồi quy Random Forest (Random Forest Regression)

#### Hồi quy Random Forest

Hồi quy Random Forest là thuật toán học có giám sát dùng **tập hợp** (ensemble) nhiều cây quyết định để dự đoán biến mục tiêu liên tục.

#### Bagging

Kỹ thuật **bagging**: chọn ngẫu nhiên các tập con dữ liệu huấn luyện để xây từng cây nhỏ; các mô hình nhỏ được kết hợp thành Random Forest và xuất ra **một** giá trị dự đoán. Cách này giúp tăng độ chính xác và **giảm phương sai** nhờ kết hợp dự đoán từ nhiều cây.

### 1.3.8 Hồi quy vectơ hỗ trợ (Support Vector Regression)

#### SVR

Support Vector Regression (SVR) là thuật toán học máy dùng máy vectơ hỗ trợ (SVM) để giải bài toán hồi quy. SVR có thể học quan hệ **phi tuyến** giữa dữ liệu đầu vào (feature) và đầu ra (target).



**Ưu điểm**

Xử lý được quan hệ tuyến tính và phi tuyến trong tập dữ liệu; **chịu outlier** tốt; có độ chính xác dự đoán cao.

## 1.4 Hai loại mô hình hồi quy (đơn và đa biến)

**Mô hình hồi quy đơn (Simple regression model)**

Đây là mô hình hồi quy cơ bản nhất: dự đoán được tạo từ **một** feature đơn biến (univariate) của dữ liệu.

**Mô hình hồi quy đa biến (Multiple regression model)**

Như tên gọi, trong mô hình này dự đoán được tạo từ **nhiều** feature của dữ liệu.

## 1.5 Chọn mô hình hồi quy tốt nhất

**Cách chọn mô hình**

Có thể xét các yếu tố: metric hiệu năng, độ phức tạp mô hình, khả năng giải thích (interpretability), v.v. Đánh giá mô hình bằng các metric như MSE, MAE,  $R^2$ , v.v. So sánh hiệu năng của các mô hình khác nhau (hồi quy tuyến tính, cây quyết định, random forest, v.v.) và chọn mô hình có metric cao nhất, độ phức tạp thấp nhất, và dễ giải thích nhất (trong phạm vi phù hợp bài toán).

## 1.6 Metric đánh giá hồi quy

**Metric đánh giá hồi quy**

- Mean Absolute Error (MAE): Trung bình của **giá trị tuyệt đối** hiệu giữa giá trị dự đoán và giá trị thực.
- Mean Squared Error (MSE): Trung bình của **bình phương** hiệu giữa giá trị thực và giá trị ước lượng.
- Median Absolute Error: Trung vị của hiệu tuyệt đối giữa giá trị dự đoán và giá trị thực.
- Root Mean Squared Error (RMSE): Căn bậc hai của MSE.
- $R^2$  (coefficient of determination): Điểm tốt nhất có thể là 1,0; có thể **âm** vì mô hình có thể tệ hơn mức tùy ý so với baseline đơn giản.
- MAPE (Mean Absolute Percentage Error): Dạng phần trăm tương đương của MAE.

## 1.7 Ứng dụng hồi quy trong học máy

### Ứng dụng hồi quy trong học máy

- Dự báo / phân tích dự đoán (Forecasting or Predictive analysis): Một trong những ứng dụng quan trọng của hồi quy là dự báo. Ví dụ: dự báo GDP, giá dầu, hoặc nói chung các dữ liệu định lượng thay đổi theo thời gian.
- Tối ưu hóa (Optimization): Hồi quy giúp tối ưu quy trình kinh doanh. Ví dụ: quản lý cửa hàng có thể lập mô hình thống kê để hiểu giờ cao điểm khách đến.
- Hiệu chỉnh sai lầm (Error correction): Trong kinh doanh, ra quyết định đúng quan trọng ngang với tối ưu quy trình. Hồi quy hỗ trợ ra quyết định đúng và **hiệu chỉnh** các quyết định đã triển khai.
- Kinh tế (Economics): Hồi quy là công cụ được dùng nhiều nhất trong kinh tế: dự đoán cung, cầu, tiêu dùng, đầu tư tồn kho, v.v.
- Tài chính (Finance): Công ty tài chính muốn giảm rủi ro danh mục và hiểu yếu tố ảnh hưởng khách hàng — có thể mô hình hóa bằng hồi quy.

## 1.8 Xây dựng mô hình hồi quy trong Python

### Cách xây dựng mô hình hồi quy

Có thể xây mô hình hồi quy từ đầu bằng Python; thư viện Scikit-learn cũng dùng được để xây mô hình hồi quy.

Ví dụ dưới đây xây mô hình hồi quy cơ bản: khớp một đường thẳng với dữ liệu.

### Bước 1: Import các gói Python

Để xây mô hình hồi quy bằng scikit-learn, cần import scikit-learn cùng các gói khác:

```
import numpy as np
from sklearn import linear_model
import sklearn.metrics as sm
import matplotlib.pyplot as plt
```

### Bước 2: Nạp dataset

Sau khi import, cần dataset để xây mô hình dự đoán hồi quy. Có thể lấy từ dataset của sklearn hoặc nguồn khác tùy nhu cầu.

```
from sklearn.datasets import make_regression
X, y = make_regression(n_samples=100, n_features=1, noise=15, random_state=42)
```

### Bước 3: Chia tập huấn luyện và kiểm tra

Cần kiểm thử trên dữ liệu chưa thấy, nên chia dataset thành tập huấn luyện và tập kiểm tra.

```
training_samples = int(0.6 * len(X))
testing_samples = len(X) - training_samples
X_train, y_train = X[:training_samples], y[:training_samples]
X_test, y_test = X[training_samples:], y[training_samples:]
```

### Bước 4: Huấn luyện và dự đoán

Sau khi chia dữ liệu, xây mô hình bằng `LinearRegression()` của Scikit-learn:

```
reg_linear = linear_model.LinearRegression()
reg_linear.fit(X_train, y_train)
y_test_pred = reg_linear.predict(X_test)
```

### Bước 5: Vẽ đồ thị

Sau dự đoán, có thể vẽ và trực quan hóa:

```
plt.scatter(X_test, y_test, color='red')
plt.plot(X_test, y_test_pred, color='black', linewidth=2)
plt.xticks(())
plt.yticks(())
plt.show()
```

### Bước 6: Tính hiệu năng

Tính hiệu năng mô hình bằng các metric:

```
print("Regressor model performance:")
print("Mean absolute error(MAE) =",
      round(sm.mean_absolute_error(y_test, y_test_pred), 2))
print("Mean squared error(MSE) =",
      round(sm.mean_squared_error(y_test, y_test_pred), 2))
print("Median absolute error =",
      round(sm.median_absolute_error(y_test, y_test_pred), 2))
print("Explain variance score =",
      round(sm.explained_variance_score(y_test, y_test_pred), 2))
print("R2 score =",
      round(sm.r2_score(y_test, y_test_pred), 2))
```

#### Output mẫu:

```
Regressor model performance:
Mean absolute error(MAE) = 1.78
Mean squared error(MSE) = 3.89
Median absolute error = 2.01
Explain variance score = -0.09
```

R2 score = -0.09

## 2 Hồi quy tuyến tính (Linear Regression)

### Hồi quy tuyến tính

Trong học máy, hồi quy tuyến tính là mô hình thống kê phân tích **quan hệ tuyến tính** giữa biến phụ thuộc và một tập biến độc lập cho trước. Quan hệ tuyến tính nghĩa là: khi một hoặc nhiều biến độc lập thay đổi (tăng/giảm), biến phụ thuộc cũng thay đổi tương ứng.

### Ứng dụng trong học máy

Dự đoán giá trị số liên tục từ quan hệ tuyến tính đã học trên dữ liệu mới; dùng trong mô hình dự báo, dự báo tài chính, đánh giá rủi ro.

### 2.1 Khái niệm chi tiết

#### Hồi quy tuyến tính (định nghĩa mở rộng)

Kỹ thuật thống kê ước lượng quan hệ tuyến tính giữa biến phụ thuộc và một hoặc nhiều biến độc lập. Trong học máy, hồi quy tuyến tính là **học có giám sát**: tập có nhãn gồm feature (đầu vào) và target (đầu ra); feature là biến độc lập, target là biến phụ thuộc.

#### Ví dụ một feature — diện tích và giá nhà

Để đơn giản, xét dữ liệu một feature và một target:

| Diện tích (ft <sup>2</sup> ) $X$ | Giá nhà $Y$ |
|----------------------------------|-------------|
| 1300                             | 240         |
| 1500                             | 320         |
| 1700                             | 330         |
| 1830                             | 295         |
| 1550                             | 256         |
| 2350                             | 409         |
| 1450                             | 319         |

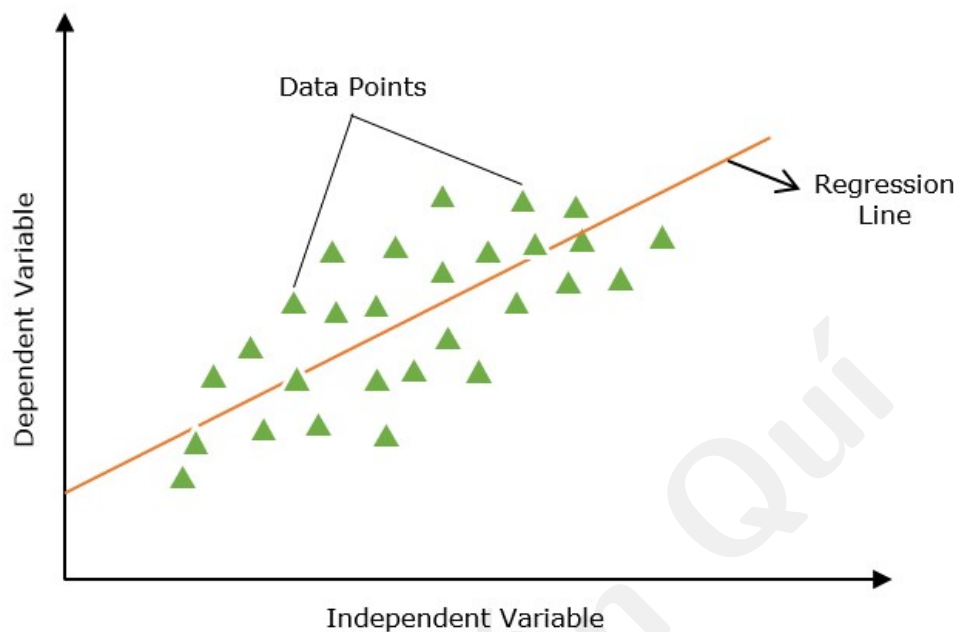
Feature **Square Feet** dùng để dự đoán target **House Price**. Biến độc lập còn gọi predictor; biến phụ thuộc còn gọi response.

#### Định nghĩa trong học máy

Hồi quy tuyến tính dùng phương trình tuyến tính mô hình hóa quan hệ giữa biến phụ thuộc  $Y$  và một hoặc nhiều biến độc lập. Mục tiêu chính: tìm **đường thẳng khớp tốt nhất** (đường hồi quy) qua các điểm dữ liệu.

### Đường hồi quy (Line of Regression)

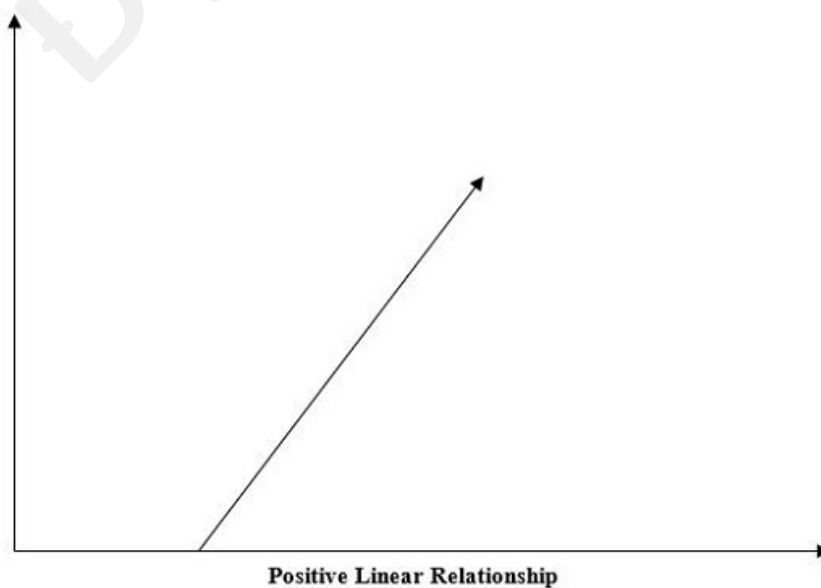
Đường thẳng thể hiện quan hệ giữa biến phụ thuộc và biến độc lập.



## 2.2 Quan hệ tuyến tính dương và âm

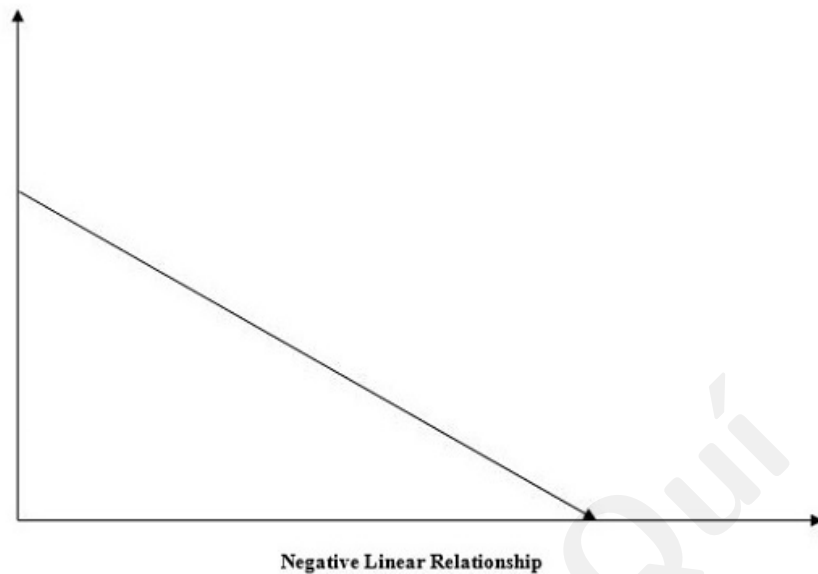
### Quan hệ tuyến tính dương (Positive Linear Relationship)

Quan hệ được gọi là **dương** khi biến độc lập và biến phụ thuộc **cùng tăng** (hoặc cùng giảm).



### Quan hệ tuyến tính âm (Negative Linear Relationship)

Quan hệ được gọi là **âm** khi biến độc lập tăng và biến phụ thuộc **giảm**.

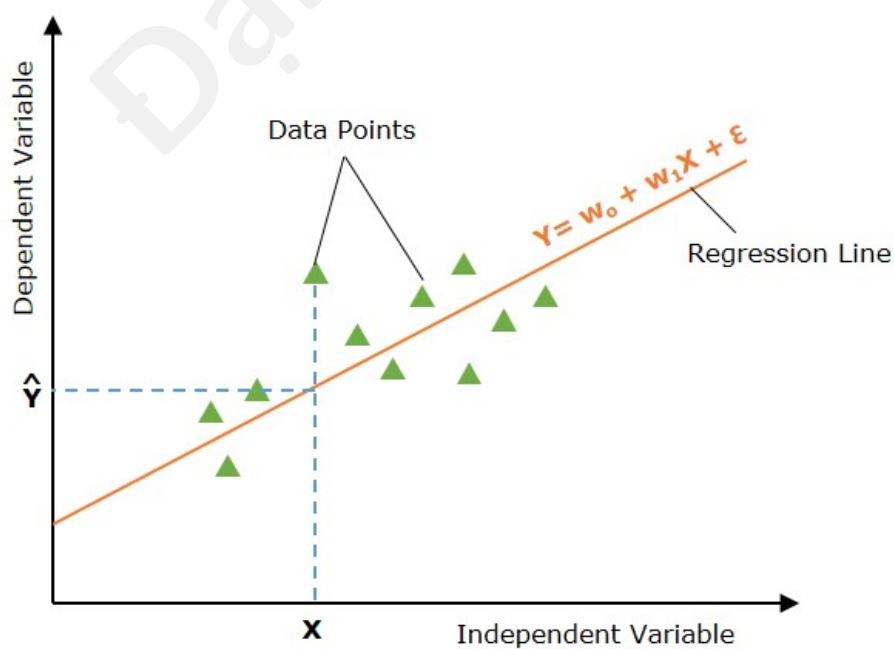


## 2.3 Hai loại hồi quy tuyến tính

### 2.3.1 Hồi quy tuyến tính đơn (Simple Linear Regression)

#### Hồi quy tuyến tính đơn

Phân tích hồi quy dùng **một** biến độc lập (predictor) để dự đoán biến phụ thuộc; mô hình hóa quan hệ tuyến tính giữa hai biến.



**Phương trình**

Đường thẳng là đường hồi quy đơn;  $\hat{Y}$  là giá trị dự đoán,  $X$  là đầu vào:

$$Y = w_0 + w_1X + \epsilon$$

- $Y$ : biến phụ thuộc (target);
- $X$ : biến độc lập (feature);
- $w_0$ : hệ số chặn (y-intercept);
- $w_1$ : độ dốc, ảnh hưởng của  $X$  lên  $Y$ ;
- $\epsilon$ : sai số, phần biến thiên của  $Y$  không giải thích bởi  $X$ .

**2.3.2 Hồi quy tuyến tính đa biến (Multiple Linear Regression)****Hồi quy tuyến tính đa biến**

Mở rộng hồi quy đơn: dự đoán response bằng **hai hoặc nhiều** feature.

**Phương trình đa biến**

$$Y = w_0 + w_1X_1 + w_2X_2 + \dots + w_pX_p + \epsilon$$

- $X_1, \dots, X_p$ : biến độc lập (features);
- $w_0, w_1, \dots, w_p$ : hệ số;
- $\epsilon$ : sai số.

**2.4 Cách hồi quy tuyến tính hoạt động****Mục tiêu**

Tìm đường khớp tốt nhất sao cho chênh lệch giữa giá trị thực và dự đoán là nhỏ nhất — bằng cách ước lượng các tham số  $w_0, w_1, \dots$ .

**Ba bước chính**

1. **Giả thuyết (Hypothesis)**: giả định quan hệ tuyến tính giữa đầu vào và đầu ra.
2. **Hàm mất mát (Cost Function)**: định nghĩa hàm mất mát đo sai số dự đoán; nhận giá trị dự đoán và thực tế, trả về một số vô hướng.
3. **Tối ưu (Optimization)**: cập nhật tham số để **tối thiểu** hóa cost; lặp đến khi lỗi đủ nhỏ.

## 2.5 Hàm giả thuyết

### Hàm giả thuyết $h_w(X)$

Giả định quan hệ tuyến tính giữa feature  $X$  và giá trị dự đoán  $\hat{Y}$ . Hàm giả thuyết trả về dự đoán cho đầu vào cho trước; thường ký hiệu  $h_w(X) = \hat{Y}$ .

### Công thức

Hồi quy đơn:

$$\hat{Y} = w_0 + w_1 X$$

Hồi quy đa biến:

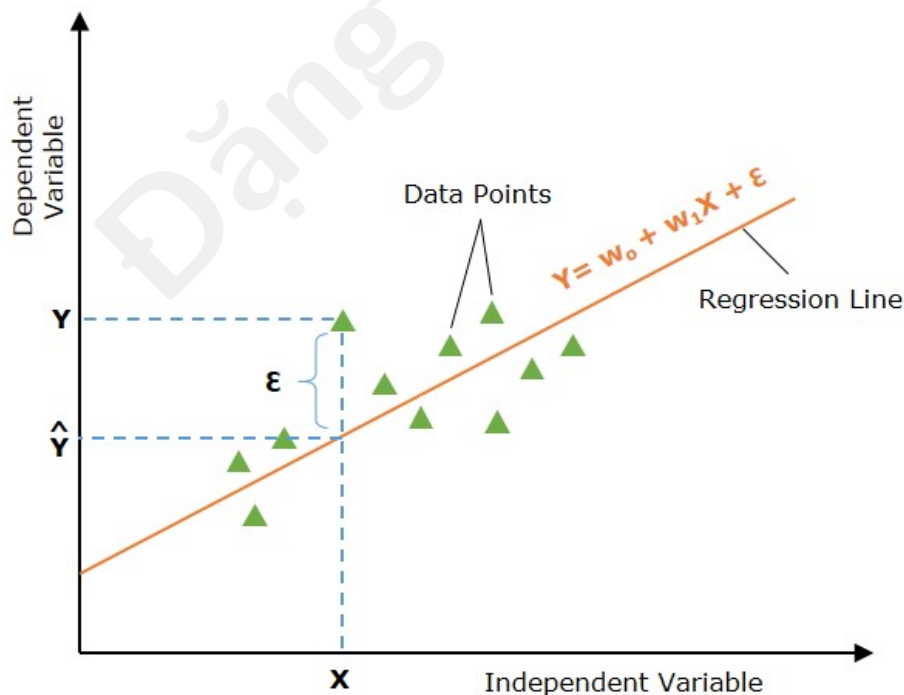
$$\hat{Y} = w_0 + w_1 X_1 + w_2 X_2 + \dots + w_p X_p$$

Với mỗi bộ trọng số khác nhau có một đường hồi quy khác; mục tiêu là chọn đường khớp tốt nhất.

## 2.6 Đường khớp tốt nhất (Finding the Best Fit Line)

### Tiêu chí đường tốt nhất

Không phải mọi đường hồi quy đều tối ưu. Đường được coi là khớp tốt nhất khi **sai số** giữa giá trị thực và dự đoán là nhỏ nhất. Sai số  $\epsilon$  tính trên mọi điểm; mục tiêu là giảm trung bình lỗi. Có thể dùng MSE, MAE, L1 loss, L2 loss, v.v.





**Hàm mất mát**

Để giảm sai số, cần hàm **cost function** / **loss function**: nhận dự đoán và thực tế, trả về một giá trị đại diện cho “giá” của dự đoán; mục tiêu là **tối thiểu hóa** hàm này.

**2.7 Hàm mất mát cho hồi quy tuyến tính****Mean Squared Error (MSE) — phổ biến nhất**

$$J(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

- $n$ : số điểm dữ liệu;
- $Y_i$ : giá trị quan sát tại điểm  $i$ ;
- $\hat{Y}_i = w_0 + w_1 X_i$ : giá trị dự đoán tại điểm  $i$ .

**2.8 Gradient descent cho tối ưu****Tối ưu mô hình**

Sau khi định nghĩa loss, bước tiếp theo là tối thiểu hóa để tìm tham số tối ưu — quá trình **model optimization**. Gradient Descent là một trong các kỹ thuật tối ưu phổ biến nhất cho hồi quy tuyến tính, đặc biệt với tập dữ liệu lớn: cập nhật tham số theo hướng giảm nhanh nhất của cost.

**Cập nhật tham số**

$$w_0 \leftarrow w_0 - \alpha \frac{\partial J}{\partial w_0}, \quad w_1 \leftarrow w_1 - \alpha \frac{\partial J}{\partial w_1}$$

với  $\alpha$  là **learning rate**. Đạo hàm riêng:

$$\frac{\partial J}{\partial w_0} = -\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i), \quad \frac{\partial J}{\partial w_1} = -\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i) X_i$$

Lặp cập nhật đến khi  $w_0, w_1$  thay đổi không đáng kể (hội tụ).

**2.9 Giả định của hồi quy tuyến tính****Giả định mô hình**

- **Đa cộng tuyến (Multi-collinearity)**: giả định ít hoặc không có đa cộng tuyến — feature phụ thuộc lẫn nhau.
- **Tự tương quan (Auto-correlation)**: giả định ít hoặc không tự tương quan — phần dư phụ thuộc lẫn nhau.

- **Quan hệ tuyến tính:** quan hệ giữa response và feature phải **tuyến tính**.

Vì phạm giả định có thể làm ước lượng chệch hoặc kém hiệu quả; cần kiểm tra giả định để đảm bảo độ chính xác.

## 2.10 Metric đánh giá

### $R^2$ (R-squared)

Do tỷ lệ phương sai biến phụ thuộc có thể dự đoán từ biến độc lập:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

### MSE

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

### RMSE

$$\text{RMSE} = \sqrt{\text{MSE}}.$$

### MAE

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

## 2.11 Ứng dụng hồi quy tuyến tính

### Mô hình dự báo (Predictive Modeling)

Dùng rộng rãi cho dự báo; ví dụ bất động sản: dự đoán giá nhà từ diện tích, vị trí, số phòng ngủ.

### Chọn feature (Feature Selection)

Trong hồi quy đa biến, phân tích hệ số giúp chọn feature; hệ số nhỏ hoặc bằng 0 có thể bỏ để đơn giản hóa mô hình.

### Dự báo tài chính (Financial Forecasting)

Dự đoán giá cổ phiếu, chỉ số kinh tế, xu hướng thị trường; hỗ trợ đầu tư và lập kế hoạch tài chính.

**Quản trị rủi ro (Risk Management)**

Mô hình hóa quan hệ giữa yếu tố rủi ro và chỉ số tài chính; ví dụ bảo hiểm: đặc điểm chủ hợp đồng và số tiền bồi thường.

**2.12 Ưu điểm hồi quy tuyến tính****Advantages**

- **Dễ giải thích (Interpretability):** dễ giải thích cách mô hình ra quyết định.
- **Tốc độ (Speed):** huấn luyện nhanh hơn nhiều thuật toán khác.
- **Phân tích dự đoán:** nền tảng cơ bản cho predictive analytics.
- **Quan hệ tuyến tính:** phương pháp mạnh khi quan hệ gần tuyến tính.
- **Đơn giản:** dễ triển khai và diễn giải.
- **Hiệu quả tính toán (Efficiency):** tính toán nhẹ.

**2.13 Thách thức thường gặp****Overfitting**

Mô hình tốt trên tập huấn luyện nhưng kém khái quát hóa trên tập kiểm tra; dự đoán kém trên dữ liệu mới.

**Đa cộng tuyến (Multicollinearity)**

Khi các biến predictor tương quan cao, ước lượng hệ số có thể không ổn định.

**Outlier**

Outlier có thể khiến đường hồi quy kém khớp phần lớn dữ liệu.

**2.14 Hồi quy đa thức — phương án thay thế****Polynomial Regression**

Quan hệ giữa biến độc lập và phụ thuộc được mô hình bằng đa thức bậc  $n$ , vượt qua giới hạn tuyến tính của hồi quy đơn và đa biến thuần túy.

**Khi nào cần**

Khi dữ liệu phi tuyến, hồi quy tuyến tính không bắt được pattern con.

### 3 Hồi quy tuyến tính đơn (Simple Linear Regression)

#### Hồi quy tuyến tính đơn

Phương pháp thống kê và học có giám sát: dùng **một** biến độc lập (predictor) để dự đoán biến phụ thuộc; mô hình hóa quan hệ tuyến tính giữa hai biến.

#### Liên hệ với hồi quy tuyến tính

Hồi quy tuyến tính đơn là một loại hồi quy tuyến tính. Thuật toán hồi quy tuyến tính với **một** feature  $\Rightarrow$  hồi quy đơn; với **nhiều** feature  $\Rightarrow$  hồi quy đa biến.

#### 3.1 Biến độc lập và phụ thuộc

##### Biến độc lập (Independent Variable)

Feature trong dataset; trong hồi quy đơn chỉ có **một** biến độc lập. Còn gọi predictor — dùng để dự đoán target; vẽ trên trục **ngang**.

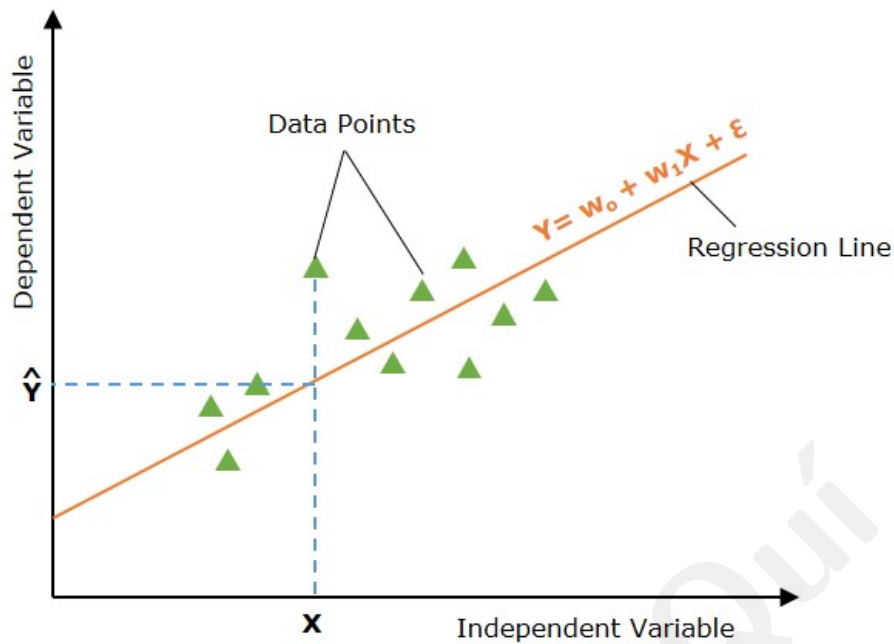
##### Biến phụ thuộc (Dependent Variable)

Giá trị target trong dataset; còn gọi response hoặc predicted variable; vẽ trên trục **đọc**.

##### Đường hồi quy (Line of Regression)

Trong hồi quy đơn, đường hồi quy là đường thẳng khớp tốt nhất các điểm, thể hiện quan hệ giữa biến phụ thuộc và độc lập.

### 3.2 Biểu diễn đồ thị



### 3.3 Mô hình toán học

#### Phương trình

$$Y = w_0 + w_1 X + \epsilon$$

- $Y$ : biến phụ thuộc (target);
- $X$ : biến độc lập (feature);
- $w_0$ : hệ số chặn;
- $w_1$ : độ dốc, ảnh hưởng của  $X$  lên  $Y$ ;
- $\epsilon$ : sai số.

### 3.4 Cách hồi quy tuyến tính đơn hoạt động

#### Mục tiêu

Tìm đường thẳng khớp tốt nhất qua các điểm sao cho chênh lệch giữa giá trị thực và dự đoán là nhỏ nhất.

#### Hàm giả thuyết

Giả thuyết: quan hệ tuyến tính giữa target (đầu ra) và feature (đầu vào):

$$\hat{Y} = w_0 + w_1 X$$

Với mỗi cặp  $(w_0, w_1)$  có một đường thẳng; tập mọi đường thẳng đó gọi là **không gian giả thuyết** (hypothesis space). Mục tiêu: tìm đường khớp tốt nhất trong không gian giả thuyết.

### Tìm đường khớp tốt nhất

Định nghĩa hàm cost/loss đo chênh lệch giữa giá trị thực và dự đoán. Mô hình khởi tạo tham số (mặc định), dùng đường hồi quy ban đầu để dự đoán trên đầu vào.

### Hàm mất mát (MSE)

Dùng giá trị đầu vào và dự đoán để tính loss; loss giúp tìm tham số tối ưu. Có thể dùng MSE, MAE,  $R^2$ , v.v.; phổ biến nhất là **mean squared error**:

$$J(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

### Tối ưu hóa

Tham số tối ưu là bộ giá trị **tối thiểu hóa** cost; quá trình lặp cập nhật tham số. Gradient Descent là kỹ thuật tối ưu đơn giản và phổ biến nhất trong hồi quy đơn. Phương trình tuyến tính với tham số tối ưu là đường hồi quy cuối cùng, dùng dự đoán dữ liệu mới.

## 3.5 Giả định

### Giả định mô hình hồi quy đơn

- **Tuyến tính (Linearity)**: biến phụ thuộc thay đổi tuyến tính theo biến độc lập; scatter plot gần đường thẳng.
- **Phương sai đồng nhất (Homoskedasticity)**: phương sai phần dư gần như nhau cho mọi quan sát.
- **Độc lập (Independence)**: các cặp  $(X, Y)$  độc lập; kiểm tra bằng scatter phần dư so với fitted values.
- **Chuẩn phần dư (Normality)**: phần dư (thực – dự đoán) gần phân phối chuẩn; kiểm tra histogram phần dư.

## 3.6 Triển khai bằng Python

### Dataset và mục đích

Hai biến: YearsExperience (độc lập) và Salary (phụ thuộc). File data/Salary\_Data.csv, 30 mẫu (bảng đầy đủ trong file CSV). Mục đích: xác định **đường thẳng nào** biểu diễn tốt nhất quan hệ giữa hai biến.

**Bảng dữ liệu (trích)**

| Năm KN | Lương  |
|--------|--------|
| 1.1    | 39343  |
| 1.3    | 46205  |
| ...    | ...    |
| 10.5   | 121872 |

**Bước 1: Chuẩn bị dữ liệu (Data Preparation)**

Tiền xử lý là bước đầu. Import thư viện trước khi nạp dữ liệu:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset = pd.read_csv('data/Salary_Data.csv')
```

Tách feature và target (YearsExperience = độc lập, Salary = phụ thuộc; bài gốc đảo nhằm X/Y):

```
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values
print(dataset.head())
```

Output mẫu head():

|   | YearsExperience | Salary  |
|---|-----------------|---------|
| 0 | 1.1             | 39343.0 |
| 1 | 1.3             | 46205.0 |
| 2 | 1.5             | 37731.0 |
| 3 | 2.0             | 43525.0 |
| 4 | 2.2             | 39891.0 |

**Kiểm tra tuyến tính của dataset**

```
plt.scatter(X, y, color="green")
plt.title("Salary vs Experience")
plt.xlabel("Years of Experience")
plt.ylabel("Salary (INR)")
plt.show()
```



### Kết luận từ biểu đồ

Biến phụ thuộc và độc lập có quan hệ gần tuyến tính  $\Rightarrow$  có thể áp dụng hồi quy tuyến tính đơn.

### Chia tập huấn luyện và kiểm tra

30 quan sát: 80% huấn luyện (24), 20% kiểm tra (6); một tập train, một tập test.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)
```

X\_train, y\_train: feature và target tập huấn luyện.

### Bước 2: Huấn luyện mô hình

Dùng LinearRegression của scikit-learn:

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

fit() huấn luyện regressor: học quan hệ giữa X\_train và y\_train.

### Bước 3: Kiểm thử mô hình

Dự đoán trên tập kiểm tra:



```
y_pred = regressor.predict(X_test)
df = pd.DataFrame({'Actual Values': y_test,
                   'Predicted Values': y_pred})
print(df)
```

$X_{test}$ : feature kiểm tra;  $y_{pred}$ : target dự đoán. Có thể dự đoán tương tự trên tập huấn luyện bằng `predict(X_train)`.

#### Bước 4: Đánh giá mô hình

Dùng MSE, RMSE, MAE,  $R^2$ :

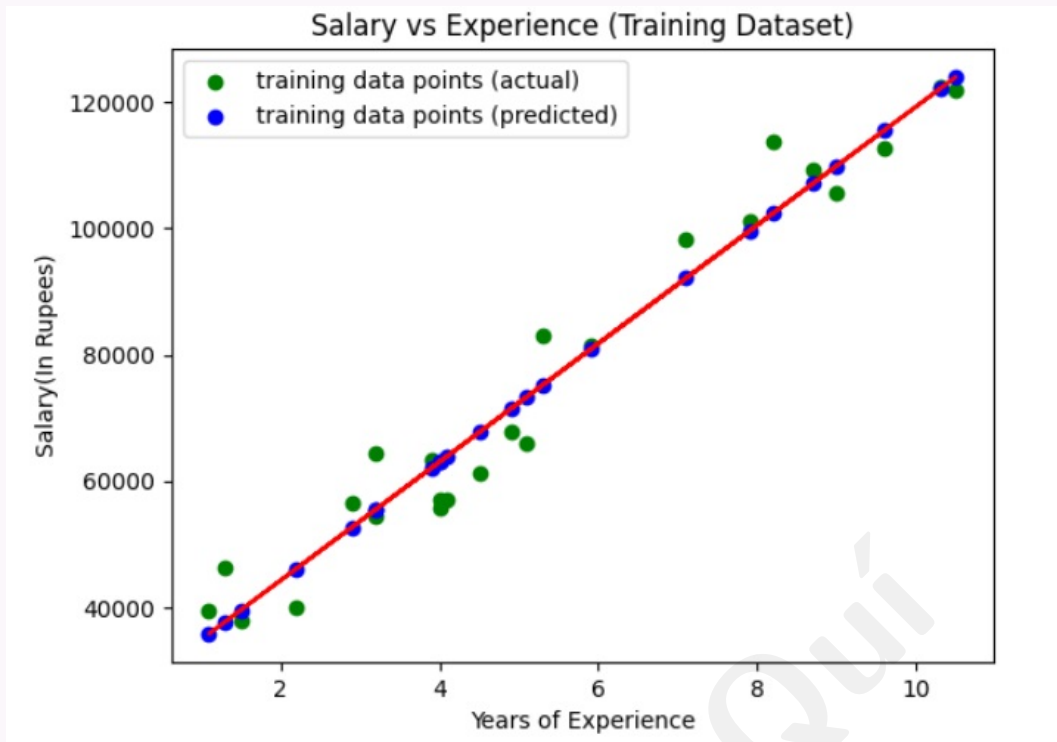
```
from sklearn.metrics import mean_squared_error, mean_absolute_error,
    r2_score
import numpy as np

y_pred = regressor.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2:", r2_score(y_test, y_pred))
```

#### Bước 5: Trực quan tập huấn luyện

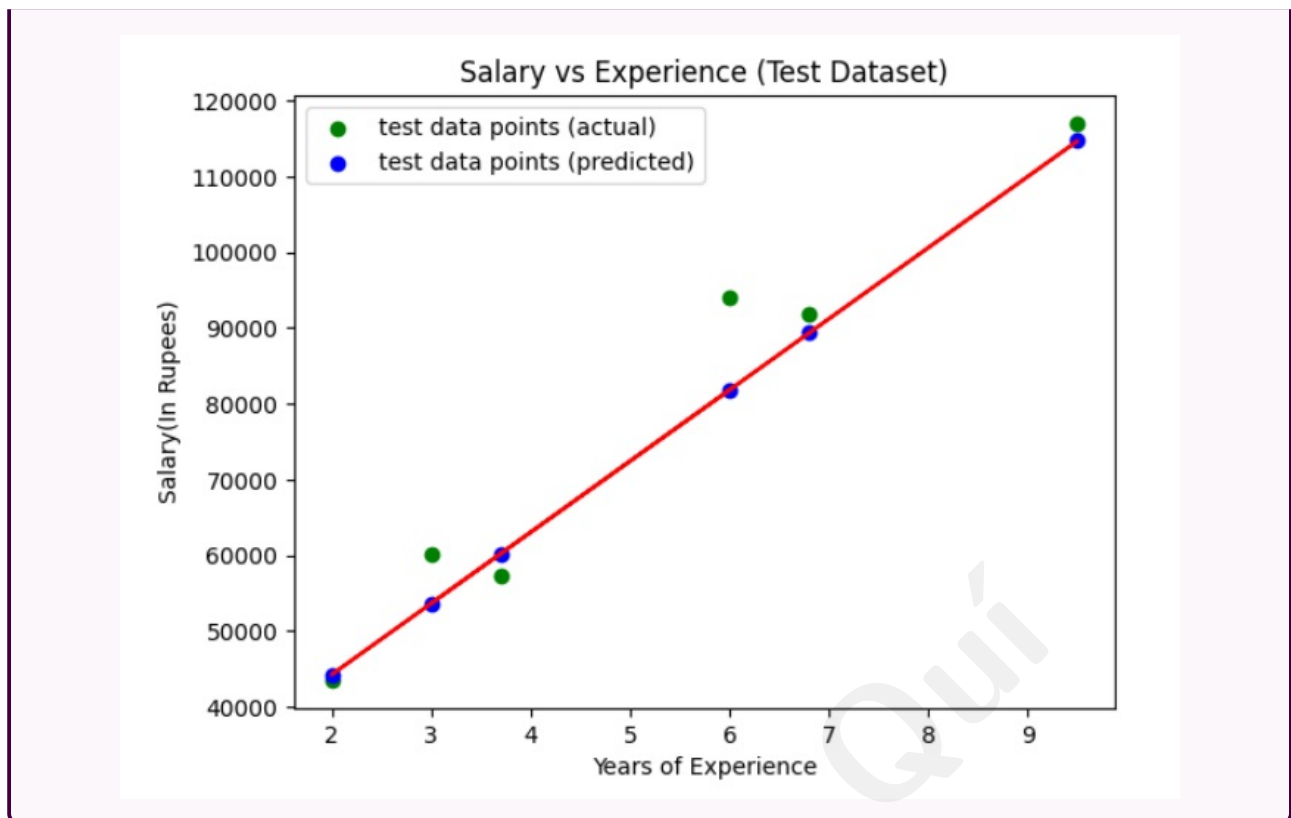
Scatter giá trị thực (xanh) và dự đoán (xanh dương); đường đỏ là đường hồi quy:

```
y_pred_train = regressor.predict(X_train)
plt.scatter(X_train, y_train, color="green",
            label="training data points (actual)")
plt.scatter(X_train, y_pred_train, color="blue",
            label="training data points (predicted)")
plt.plot(X_train, y_pred_train, color="red")
plt.title("Salary vs Experience (Training Dataset)")
plt.xlabel("Years of Experience")
plt.ylabel("Salary (INR)")
plt.legend()
plt.show()
```



#### Bước 6: Trực quan tập kiểm tra

```
y_pred = regressor.predict(X_test)
plt.scatter(X_test, y_test, color="green",
            label="test data points (actual)")
plt.scatter(X_test, y_pred, color="blue",
            label="test data points (predicted)")
plt.plot(X_test, y_pred, color="red")
plt.title("Salary vs Experience (Test Dataset)")
plt.xlabel("Years of Experience")
plt.ylabel("Salary (INR)")
plt.legend()
plt.show()
```



## 4 Hồi quy tuyến tính đa biến (Multiple Linear Regression)

### Hồi quy tuyến tính đa biến (MLR)

Thuật toán học có giám sát mô hình quan hệ giữa **một** biến phụ thuộc và **nhiều** biến độc lập; quan hệ này dùng để dự đoán giá trị biến phụ thuộc.

### Hai loại hồi quy tuyến tính

- **Hồi quy đơn:** một biến phụ thuộc và một biến độc lập.
- **Hồi quy đa biến:** một biến phụ thuộc và **hơn một** biến độc lập.

### 4.1 Khái niệm chi tiết

#### MLR trong học máy

Kỹ thuật thống kê dự đoán kết quả biến phụ thuộc từ giá trị nhiều biến độc lập. Thuật toán huấn luyện trên dữ liệu để học quan hệ (đường hồi quy) khớp tốt nhất; quan hệ mô tả các yếu tố ảnh hưởng kết quả và dùng để **dự báo** khi biết giá trị các biến độc lập.

**Kiểu biến**

Trong hồi quy tuyến tính (đơn và đa biến), biến phụ thuộc là **liên tục** (số). Biến độc lập có thể liên tục hoặc rời rạc (số); biến phân loại (giới tính, nghề nghiệp) cần **mã hóa số** trước khi đưa vào mô hình.

**Mở rộng hồi quy đơn**

MLR mở rộng hồi quy đơn: dự đoán bằng hai hoặc nhiều feature.

**4.2 Phương trình****Mô hình với  $n$  quan sát,  $p$  feature**

Đường hồi quy cho  $p$  feature:

$$h(x_i) = w_0 + w_1x_{i1} + w_2x_{i2} + \dots + w_px_{ip}$$

$h(x_i)$  là giá trị dự đoán;  $w_0, w_1, \dots, w_p$  là hệ số hồi quy.

**Phần dư (residual error)**

Mô hình MLR luôn gồm sai số trong dữ liệu:

$$y_i = w_0 + w_1x_{i1} + \dots + w_px_{ip} + e_i$$

hoặc  $y_i = h(x_i) + e_i$ , tức  $e_i = y_i - h(x_i)$ .

**4.3 Giả định****Giả định MLR (chi tiết)**

1. **Tuyến tính**: quan hệ tuyến tính giữa target và predictor.
2. **Độc lập**: mỗi quan sát độc lập; giá trị target của quan sát này không phụ thuộc quan sát khác.
3. **Phương sai đồng nhất (Homoscedasticity)**: phương sai phần dư tương tự trên miền giá trị mỗi biến độc lập.
4. **Chuẩn phần dư**: phần dư (thực – dự đoán) gần phân phối chuẩn.
5. **Không đa cộng tuyến**: biến độc lập không tương quan cao lẫn nhau.
6. **Không tự tương quan phần dư**: phần dư độc lập lẫn nhau.
7. **Biến độc lập cố định**: giá trị biến độc lập cố định trong các mẫu lặp.

Vi phạm giả định có thể làm ước lượng chệch hoặc kém hiệu quả; cần kiểm tra giả định.

## 4.4 Triển khai bằng Python (Scikit-learn)

### Lớp LinearRegression

Dùng cùng lớp LinearRegression như hồi quy đơn, nhưng đầu vào là **ma trận nhiều cột** feature.

### Bước 1: Chuẩn bị dữ liệu

Import thư viện trước khi nạp dữ liệu:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Nạp data.csv, tách  $X$  và  $y$ :

```
dataset = pd.read_csv('data/data.csv')
X = dataset[['R&D Spend', 'Administration', 'Marketing Spend']]
y = dataset['Profit']
```

Xem 5 dòng đầu:

```
print(X.head())
print(y.head())
```

Output X.head():

|     | R&D Spend | Administration | Marketing Spend |
|-----|-----------|----------------|-----------------|
| 0   | 165349.20 | 136897.80      | 471784.10       |
| 1   | 162597.70 | 151377.59      | 443898.53       |
| ... |           |                |                 |

Output y.head():

|     |           |
|-----|-----------|
| 0   | 192261.83 |
| 1   | 191792.06 |
| ... |           |

### Chia train/test

20% kiểm tra: 40 mẫu train, 10 mẫu test (trong 50 quan sát).

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)
```

$X_{\text{train}}$ ,  $X_{\text{test}}$ : feature;  $y_{\text{train}}$ ,  $y_{\text{test}}$ : target.

### Bước 2: Huấn luyện

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

`fit()` học quan hệ giữa `X_train` và `y_train`.

### Bước 3: Kiểm thử

```
y_pred = regressor.predict(X_test)
df = pd.DataFrame({'Real Values': y_test.values,
                  'Predicted Values': y_pred})
print(df)
```

Output mẫu:

|     | Real Values | Predicted Values |
|-----|-------------|------------------|
| 23  | 108733.99   | 110159.827849    |
| 43  | 69758.98    | 59787.885207     |
| ... |             |                  |

So sánh giá trị thực và dự đoán.

### Bước 4: Đánh giá mô hình

```
from sklearn.metrics import mean_squared_error, mean_absolute_error,
    r2_score
import numpy as np

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("Mean Absolute Error (MAE):", mae)
print("R-squared (R2):", r2)
```

Output mẫu:

```
Mean Squared Error (MSE): 72684687.6336162
Root Mean Squared Error (RMSE): 8525.531516193943
Mean Absolute Error (MAE): 6425.118502810154
R-squared (R2): 0.9588459519573707
```

**Giải thích  $R^2 \approx 0,96$**

Khoảng 96% biến thiên của Profit được giải thích bởi các biến đầu vào (trên tập kiểm tra của ví dụ gốc).

**Bước 5: Dự đoán dữ liệu mới**

Dự đoán Profit khi R&D = 166343.2, Administration = 136787.8, Marketing = 461724.1:

```
new_data = [[166343.2, 136787.8, 461724.1]]
profit = regressor.predict(new_data)
print(profit)
```

**Output:** [193053.61874652].

**Tham số mô hình (intercept và hệ số)**

Intercept và coefficients mô tả quan hệ giữa biến phụ thuộc và các biến độc lập:

$$Y = w_0 + w_1X_1 + w_2X_2 + w_3X_3$$

$X_1$  = R&D Spend;  $X_2$  = Administration;  $X_3$  = Marketing Spend.

**In hệ số và chặn**

```
print("coefficients:", regressor.coef_)
print("intercept:", regressor.intercept_)
```

**Output:**

```
coefficients: [ 0.81129358 -0.06184074  0.02515044]
intercept: 54946.94052163202
```

**Giải thích hệ số**

- $w_0 = 54946.94$
- $w_1 = 0.8113$  (R&D): tăng 1 USD R&D  $\Rightarrow$  Profit tăng khoảng 0.81 USD.
- $w_2 = -0.0618$  (Administration): tăng 1 USD  $\Rightarrow$  Profit giảm khoảng 0.06 USD.
- $w_3 = 0.0252$  (Marketing): tăng 1 USD  $\Rightarrow$  Profit tăng khoảng 0.025 USD.

**Kiểm chứng tay**

Profit =  $54946.94 + 0.8113 \times 166343.2 - 0.0618 \times 136787.8 + 0.0252 \times 461724.1 \approx 193053.62$   
 Gần `predict(new_data)`; chênh lệch nhỏ do **phần dư**: residual  $\approx 193053.6187 - 193053.6163 \approx 0.0025$ .

## 4.5 Ứng dụng

### Ứng dụng MLR

| Lĩnh vực        | Mô tả                                                 |
|-----------------|-------------------------------------------------------|
| Tài chính       | Dự đoán giá cổ phiếu, tỷ giá, đánh giá tín dụng.      |
| Marketing       | Dự đoán doanh số, churn, hiệu quả chiến dịch.         |
| Bất động sản    | Giá nhà theo diện tích, vị trí, số phòng.             |
| Y tế            | Kết quả bệnh nhân, tác động điều trị, yếu tố nguy cơ. |
| Kinh tế         | Tăng trưởng, chính sách, lạm phát.                    |
| Khoa học xã hội | Hiện tượng xã hội, bầu cử, hành vi.                   |

## 4.6 Thách thức

### Thách thức MLR

| Thách thức    | Mô tả                                                                    |
|---------------|--------------------------------------------------------------------------|
| Đa cộng tuyến | Feature tương quan cao $\Rightarrow$ hệ số không ổn định, khó diễn giải. |
| Overfitting   | Khớp quá sát tập train, kém trên dữ liệu mới.                            |
| Underfitting  | Không bắt pattern, kém cả train và test.                                 |
| Phi tuyến     | MLR giả định tuyến tính; quan hệ cong $\Rightarrow$ dự đoán sai.         |
| Outlier       | Ảnh hưởng mạnh, nhất là tập nhỏ.                                         |
| Dữ liệu thiếu | Kết quả chệch, không chính xác.                                          |

## 4.7 So sánh hồi quy đơn và đa biến

### Bảng so sánh

| Tiêu chí         | Đơn biến                                        | Đa biến                                                                |
|------------------|-------------------------------------------------|------------------------------------------------------------------------|
| Số biến độc lập  | Một                                             | Hai trở lên                                                            |
| Phương trình     | $y = w_0 + w_1x$                                | $y = w_0 + \sum w_jx_j$                                                |
| Độ phức tạp      | Thấp hơn                                        | Cao hơn                                                                |
| Ứng dụng         | Giá nhà theo diện tích; doanh số theo quảng cáo | Doanh số theo giá, đối thủ; điểm sinh viên theo giờ học, điểm danh, IQ |
| Giải thích hệ số | Dễ hơn                                          | Khó hơn                                                                |



## 5 Hồi quy đa thức (Polynomial Regression)

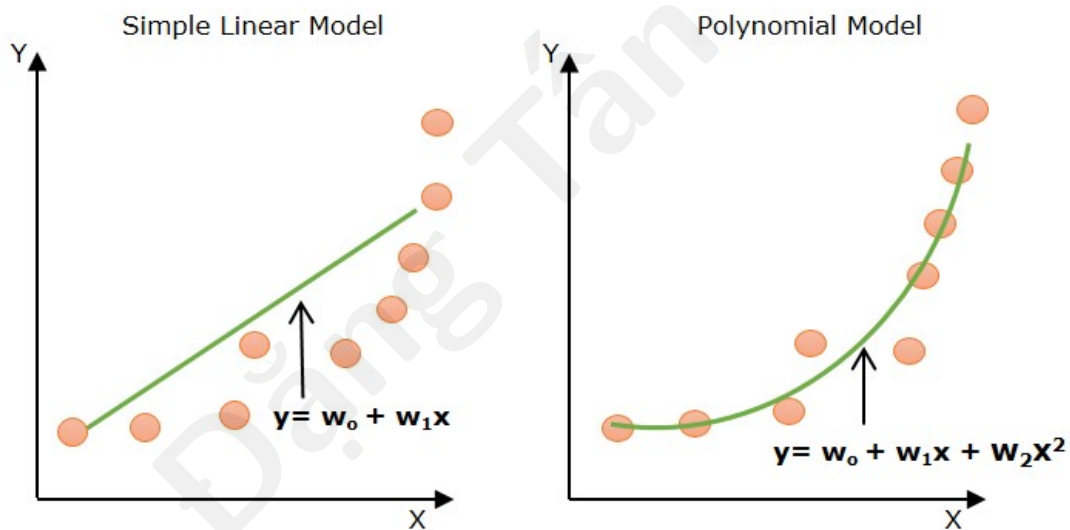
### Hồi quy đa thức

Polynomial Linear Regression mô hình hóa quan hệ giữa biến độc lập và phụ thuộc bằng **hàm đa thức bậc  $n$** , vượt qua giới hạn tuyến tính của hồi quy đơn và hồi quy tuyến tính đa biến thuần túy.

### 5.1 Vì sao cần hồi quy đa thức?

#### Chọn linear hay polynomial

Trong ML và data science, chọn hồi quy tuyến tính hay đa thức phụ thuộc **đặc tính dataset**. Dữ liệu **phi tuyến** không khớp bằng hồi quy tuyến tính; nếu ép tuyến tính, mô hình không bắt được pattern cong.



#### Đọc hình so sánh

Mô hình tuyến tính đơn khó khớp các điểm; mô hình đa thức khớp phần lớn điểm tốt hơn.

## 5.2 Phương trình mô hình

### Đa thức bậc $n$

$$y = w_0 + w_1x + w_2x^2 + w_3x^3 + \dots + w_nx^n + \epsilon$$

- $y$ : biến phụ thuộc (output);
- $x$ : biến độc lập (input);
- $w_0, w_1, \dots, w_n$ : hệ số;
- $n$ : bậc đa thức (lũy thừa cao nhất của  $x$ );
- $\epsilon$ : sai số / phần dư.

### Đa thức bậc 2 (parabol)

$$y = w_0 + w_1x + w_2x^2 + \epsilon$$

Khớp đường cong parabol với các điểm dữ liệu.

## 5.3 Cách hoạt động

### Cơ chế trong học máy

Hồi quy đa thức hoạt động tương tự hồi quy tuyến tính nhưng được mô hình như **hồi quy tuyến tính đa biến**: biến đổi feature gốc thành  $x, x^2, \dots, x^n$ ; các cột này được coi là feature độc lập riêng; huấn luyện `LinearRegression` trên không gian mở rộng.

### Khác MLR thuần

MLR giả định feature gốc tuyến tính với target; trong hồi quy đa thức, các cột  $x^k$  **phụ thuộc** cùng một  $x$  ban đầu.

## 5.4 Triển khai bằng Python (Scikit-learn)

### Thư viện

Triển khai bằng Python; dùng Scikit-learn để xây mô hình hồi quy.

### Bước 1: Chuẩn bị dữ liệu — import

Chuẩn bị dữ liệu là bước quan trọng. Dùng pandas đọc CSV, NumPy cho mảng, `PolynomialFeatures` để biến đổi bậc:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
```

### Nạp dataset

```
data = pd.read_csv('data/ice_cream_selling_data.csv')  
print(data.head())
```

### Output head():

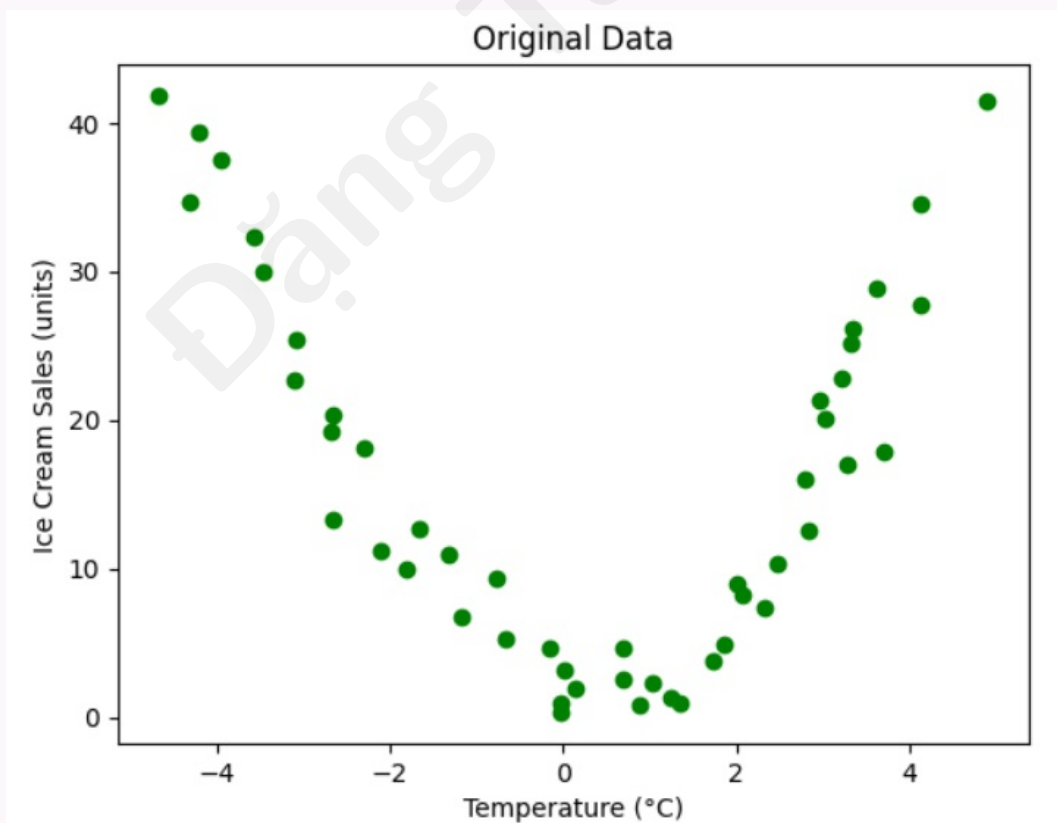
|     | Temperature | IceCreamSales |
|-----|-------------|---------------|
| 0   | -4.662263   | 41.842986     |
| 1   | -4.316559   | 34.661120     |
| ... |             |               |

### Tách X và y

```
X = data.iloc[:, 0].values.reshape(-1, 1)  
y = data.iloc[:, 1].values
```

### Trực quan dữ liệu gốc

```
plt.scatter(X, y, color="green")  
plt.title("Original Data")  
plt.xlabel("Temperature (C)")  
plt.ylabel("Ice Cream Sales (units)")  
plt.show()
```



**Nhận xét đồ thị**

Dữ liệu gần dạng **parabol** (đa thức bậc 2)  $\Rightarrow$  mô hình hóa bằng hồi quy đa thức bậc 2 phù hợp quan hệ nhiệt độ – doanh số kem.

**Tạo đặc trưng đa thức bậc 2**

```
degree = 2
poly_features = PolynomialFeatures(degree=degree)
X_poly = poly_features.fit_transform(X)
```

X\_poly: feature đa thức từ X; kích thước (49, 3) (cột hằng,  $x$ ,  $x^2$ ).

**Bước 2: Huấn luyện mô hình**

Hồi quy đa thức là trường hợp đặc biệt của hồi quy tuyến tính:

```
from sklearn.linear_model import LinearRegression
lr_model = LinearRegression()
lr_model.fit(X_poly, y)
```

**Bước 3: Dự đoán và kiểm tra**

Dự đoán trên dữ liệu hiện có trước khi dự đoán điểm mới:

```
y_pred = lr_model.predict(X_poly)
df = pd.DataFrame({'Actual Values': y, 'Predicted Values': y_pred})
print(df)
```

**Output trích (5 dòng đầu bài gốc):**

|   | Actual Values | Predicted Values |
|---|---------------|------------------|
| 0 | 41.842986     | 46.564507        |
| 1 | 34.661120     | 40.600548        |
| 2 | 39.383001     | 38.915089        |
| 3 | 37.539845     | 34.749272        |
| 4 | 32.284531     | 29.331940        |

So sánh cột thực tế và dự đoán.

**Bước 4: Đánh giá hiệu năng —  $R^2$** 

```
from sklearn.metrics import r2_score
r2 = r2_score(y, y_pred)
print(r2)
```

**Output:** 0.9321137090423877

**Giải thích  $R^2$** 

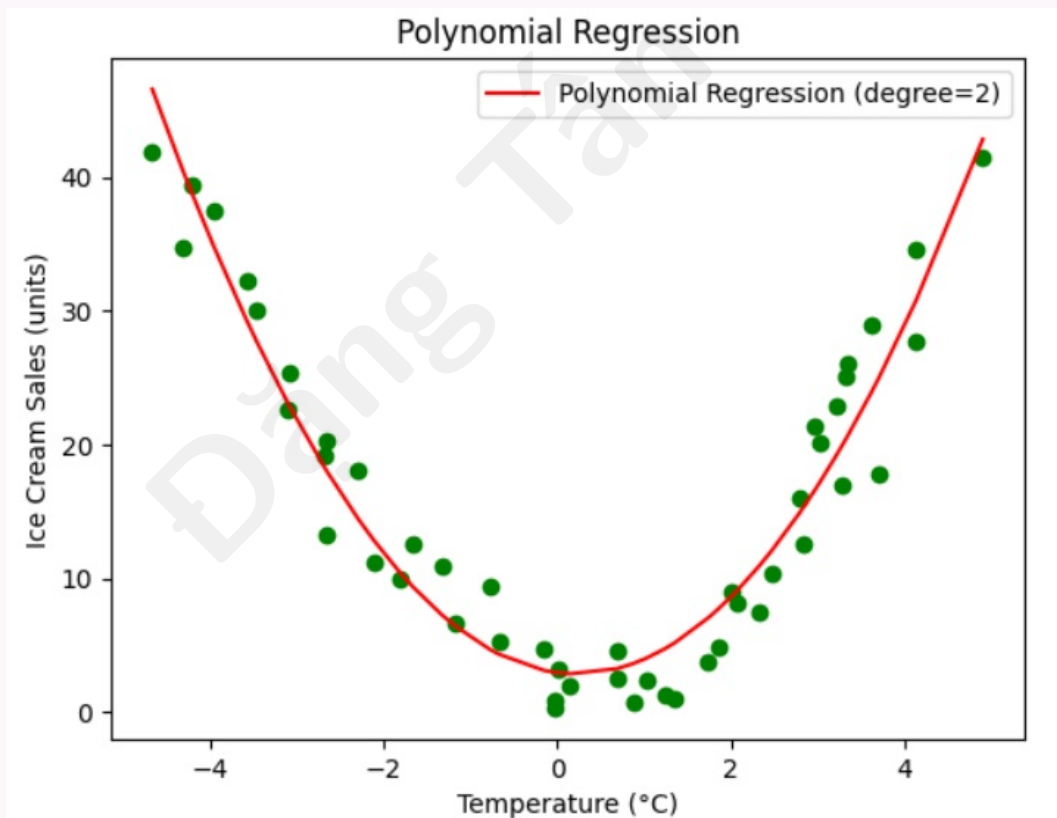
$R^2$  đo tỷ lệ phương sai biến phụ thuộc có thể dự đoán từ biến độc lập.

Điểm cao  $\Rightarrow$  khớp tốt hơn; 1 = khớp hoàn hảo; 0 = không có quan hệ tuyến tính giữa dự đoán và thực tế.

Với  $R^2 \approx 0,932$ : khoảng 93% biến thiên output được giải thích bởi input; phần lớn điểm nằm quanh đường cong hồi quy.

**Bước 5: Trục quan kết quả hồi quy đa thức**

```
plt.scatter(X, y, color="green")
plt.plot(X, y_pred, color='red',
         label=f'Polynomial Regression (degree={degree})')
plt.xlabel("Temperature (C)")
plt.ylabel("Ice Cream Sales (units)")
plt.legend()
plt.title('Polynomial Regression')
plt.show()
```

**Đọc biểu đồ**

Đường đỏ (parabol bậc 2) khớp tốt dữ liệu gốc; dùng để dự đoán; giá trị dự đoán gần giá trị thực.

**Bước 6: Dự đoán dữ liệu mới**

Nhiệt độ 1,9929 °C:

```
X_new = np.array([[1.9929]])  
X_new_poly = poly_features.transform(X_new)  
y_new_pred = lr_model.predict(X_new_poly)  
print(y_new_pred)
```

**Output:** [8.57450466] — dự đoán khoảng 8.57 đơn vị kem bán ra.

**Bậc đa thức và overfitting**

Bậc quá cao dễ **overfitting**; khi triển khai thực tế nên thử cross-validation hoặc regularization.